JAVA PROGRAMMING LABORATORY RECORD

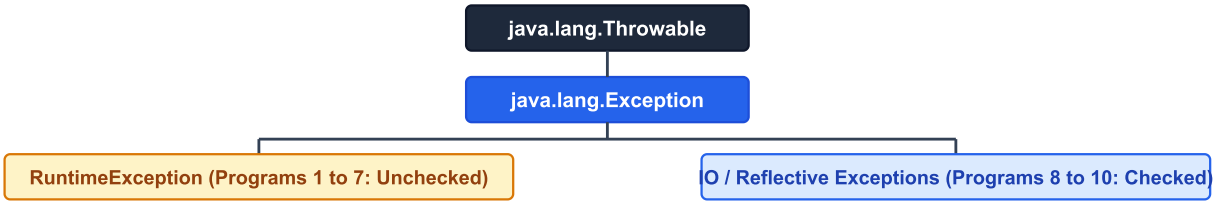
10 Exception Handling Programs

Question, Step-by-Step Process, Algorithm, UML Class Diagrams, Source Code & Sample Output

ARCHITECTURAL OVERVIEW

Exception handling in Java is a robust mechanism designed to handle runtime errors and maintain the normal application flow. Every exception and error type is a subclass of the root **java.lang.Throwable** class, which bifurcates into Checked Exceptions (enforced at compile time) and Unchecked Runtime Exceptions (occurring during execution).

JAVA EXCEPTION CLASS HIERARCHY (UML OVERVIEW)



MASTER INDEX OF LAB EXPERIMENTS

#	Exception Class	Main Demo Class	Category	Page
01	ArrayIndexOutOfBoundsException	ArrayExceptionDemo	UNCHECKED	Page 2
02	NullPointerException	NullExceptionDemo	UNCHECKED	Page 3
03	NumberFormatException	NumberFormatDemo	UNCHECKED	Page 4
04	StringIndexOutOfBoundsException	StringIndexDemo	UNCHECKED	Page 5
05	ClassCastException	ClassCastDemo	UNCHECKED	Page 6
06	IllegalArgumentException	IllegalArgumentDemo	UNCHECKED	Page 7
07	IllegalStateException	IllegalStateDemo	UNCHECKED	Page 8
08	FileNotFoundException	FileExceptionDemo	CHECKED	Page 9
09	IOException	IOExceptionDemo	CHECKED	Page 10
10	ClassNotFoundException	ClassNotFoundDemo	CHECKED	Page 11

PROGRAM 01 ArrayIndexOutOfBoundsException

JAVA.LANG.ARRAYINDEXOUTOFBOUNDSEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **ArrayIndexOutOfBoundsException** when an invalid array index is accessed.

2. STEP-BY-STEP PROCESS

1. Define the public driver class `ArrayExceptionDemo` with the standard `main()` method.
2. Declare and initialize a 1D integer array `numbers` with 5 elements (valid indices 0 to 4).
3. Open a `try` block and attempt to print the element at index 8 (`numbers[8]`).
4. Implement a `catch` block capturing `ArrayIndexOutOfBoundsException`.
5. Print the exception message to the console gracefully without crashing.

3. ALGORITHM

1. Start the program.
2. Create an integer array with some values.
3. Try to access an index outside the valid range.
4. Catch the `ArrayIndexOutOfBoundsException`.
5. Display a suitable error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
public class ArrayExceptionDemo
{
    public static void main(String[] args)
    {
        int[] numbers = {10, 20, 30, 40, 50};
        try
        {
            System.out.println("Value = " + numbers[8]);
        }
        catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Exception: Array index is out of bounds.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Array index is out of bounds.
```

PROGRAM 02

NullPointerException

JAVA.LANG.NULLPOINTEREXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **NullPointerException** when a method is called on a null String object.

2. STEP-BY-STEP PROCESS

1. Define the class `NullExceptionDemo` containing the `main()` method.
2. Declare a `String` reference variable named `name` and initialize it to `null`.
3. Inside a `try` block, invoke `name.length()` on the null reference.
4. Catch the resulting `NullPointerException` in the `catch` block.
5. Print the error message indicating that the String object is null.

3. ALGORITHM

1. Start the program.
2. Declare a String variable and assign `null`.
3. Try to call a String method on it.
4. Catch the `NullPointerException`.
5. Display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
public class NullExceptionDemo
{
    public static void main(String[] args)
    {
        String name = null;
        try
        {
            System.out.println(name.length());
        }
        catch (NullPointerException e)
        {
            System.out.println("Exception: String object is null.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: String object is null.
```

PROGRAM 03

NumberFormatException

JAVA.LANG.NUMBERFORMATEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **NumberFormatException** when a non-numeric String is converted into an integer.

2. STEP-BY-STEP PROCESS

1. Define the class `NumberFormatDemo` with the `main()` method.
2. Assign the non-numeric literal `"Java"` to a `String` variable named `value`.
3. Inside a `try` block, pass `value` to `Integer.parseInt(value)`.
4. Catch the thrown `NumberFormatException` when parsing fails.
5. Print a clear notification stating that the number format is invalid.

3. ALGORITHM

1. Start the program.
2. Store a non-numeric value in a String.
3. Try to convert it into an integer using `Integer.parseInt()`.
4. Catch the `NumberFormatException`.
5. Display an appropriate message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```
public class NumberFormatDemo
{
    public static void main(String[] args)
    {
        String value = "Java";
        try
        {
            int number = Integer.parseInt(value);
            System.out.println("Number = " + number);
        }
        catch (NumberFormatException e)
        {
            System.out.println("Exception: Invalid number format.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Invalid number format.
```

PROGRAM 04 StringIndexOutOfBoundsException

JAVA.LANG.STRINGINDEXOUTOFBOUNDSEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **StringIndexOutOfBoundsException** when an invalid String index is accessed.

2. STEP-BY-STEP PROCESS

1. Define the class `StringIndexDemo` with the `main()` method.
2. Initialize a `String` variable `text = "JAVA"` (valid character indices 0 to 3).
3. Inside a `try` block, call `text.charAt(10)` to request an out-of-range character.
4. Catch the resulting `StringIndexOutOfBoundsException`.
5. Display a user-friendly message indicating an invalid String index.

3. ALGORITHM

1. Start the program.
2. Create a String.
3. Try to access a character using an invalid index.
4. Catch the `StringIndexOutOfBoundsException`.
5. Display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```
public class StringIndexDemo
{
    public static void main(String[] args)
    {
        String text = "JAVA";
        try
        {
            System.out.println(text.charAt(10));
        }
        catch (StringIndexOutOfBoundsException e)
        {
            System.out.println("Exception: Invalid String index.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Invalid String index.
```

PROGRAM 05 ClassCastException

JAVA.LANG.CLASSCASTEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **ClassCastException** when an object is incorrectly cast to an unrelated or subclass reference.

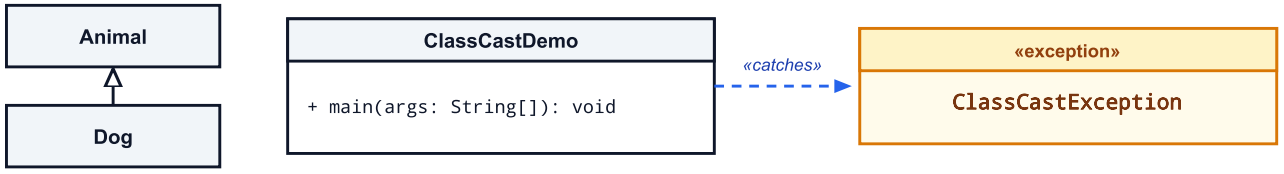
2. STEP-BY-STEP PROCESS

1. Create a base class `Animal` and a derived subclass `Dog extends Animal`.
2. In `ClassCastDemo.main()`, instantiate a plain parent object: `Animal animal = new Animal()`.
3. Inside a `try` block, attempt to downcast the parent instance to the child type: `Dog dog = (Dog) animal`.
4. Catch the `ClassCastException` thrown at runtime and print the error message.

3. ALGORITHM

1. Start the program.
2. Create an object of one class (`Animal`).
3. Store it in a parent reference.
4. Try to cast it into an incompatible subclass (Dog).
5. Catch the `ClassCastException`.
6. Display the error message and stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
class Animal { }
class Dog extends Animal { }
class ClassCastDemo
{
    public static void main(String[] args)
    {
        Animal animal = new Animal();
        try
        {
            Dog dog = (Dog) animal;
            System.out.println("Casting successful.");
        }
        catch (ClassCastException e)
        {
            System.out.println("Exception: Invalid type casting.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Invalid type casting.
```

PROGRAM 06 IllegalArgumentException

JAVA.LANG.ILLEGALARGUMENTEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **IllegalArgumentException** when an invalid argument is passed to a method.

2. STEP-BY-STEP PROCESS

1. Define a static helper method `checkAge(int age)` inside `IllegalArgumentDemo`.
2. Validate the parameter: if `age < 0`, explicitly throw a new `IllegalArgumentException`.
3. In `main()`, call `checkAge(-5)` inside a `try` block.
4. Catch the `IllegalArgumentException` and display its message via `e.getMessage()`.

3. ALGORITHM

1. Start the program.
2. Create a method that accepts a positive age.
3. Pass a negative value to the method.
4. Throw `IllegalArgumentException`.
5. Catch the exception in `main()` and display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```
public class IllegalArgumentDemo
{
    static void checkAge(int age)
    {
        if (age < 0)
        {
            throw new IllegalArgumentException("Age cannot be negative.");
        }
        System.out.println("Valid age: " + age);
    }
    public static void main(String[] args)
    {
        try
        {
            checkAge(-5);
        }
        catch (IllegalArgumentException e)
        {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Age cannot be negative.
```

PROGRAM 07 **IllegalStateException** JAVA.LANG.ILLEGALSTATEEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **IllegalStateException** when an operation is performed in an inappropriate state.

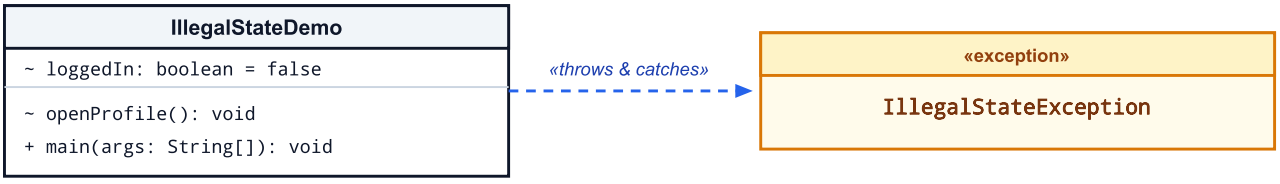
2. STEP-BY-STEP PROCESS

1. Declare a static boolean flag `loggedIn = false` in `IllegalStateDemo`.
2. Create `openProfile()` which checks `!loggedIn` and throws `IllegalStateException` if the user is not authenticated.
3. Call `openProfile()` inside a `try` block in `main()` while `loggedIn` is still `false`.
4. Catch `IllegalStateException` and print the exception message.

3. ALGORITHM

1. Start the program.
2. Create a method representing a resource.
3. Check whether the resource is available.
4. If it is unavailable, throw `IllegalStateException`.
5. Catch the exception and display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
public class IllegalStateDemo
{
    static boolean loggedIn = false;
    static void openProfile()
    {
        if (!loggedIn)
        {
            throw new IllegalStateException("User must log in first.");
        }
        System.out.println("Profile opened.");
    }
    public static void main(String[] args)
    {
        try
        {
            openProfile();
        }
        catch (IllegalStateException e)
        {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: User must log in first.
```

PROGRAM 08 FileNotFoundException

JAVA.IO.FILENOTFOUNDEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **FileNotFoundException** when the specified file does not exist.

2. STEP-BY-STEP PROCESS

1. Import `java.io.FileReader` and `java.io.FileNotFoundException`.
2. Define the class `FileExceptionDemo` with `main()`.
3. Inside a `try` block, instantiate new `FileReader("missing.txt")` for a non-existent file.
4. Catch the checked `FileNotFoundException` in the `catch` block.
5. Print the error message indicating that the file was not found.

3. ALGORITHM

1. Start the program.
2. Specify the name of a file that does not exist.
3. Try to open the file using `FileReader`.
4. Catch the `FileNotFoundException`.
5. Display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
import java.io.FileReader;
import java.io.FileNotFoundException;
public class FileExceptionDemo
{
    public static void main(String[] args)
    {
        try
        {
            FileReader file = new FileReader("missing.txt");
            System.out.println("File opened successfully.");
        }
        catch (FileNotFoundException e)
        {
            System.out.println("Exception: File not found.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: File not found.
```

PROGRAM 09

IOException

JAVA.IO.IOEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **IOException** while performing an input/output operation.

2. STEP-BY-STEP PROCESS

1. Import `java.io.IOException` at the top of the source file.
2. Define a static method `readData() throws IOException` that explicitly throws a new `IOException`.
3. Call `readData()` inside a `try` block within `main()`.
4. Catch the propagated `IOException` and display its message via `e.getMessage()`.

3. ALGORITHM

1. Start the program.
2. Create a method that performs an I/O operation.
3. Generate an `IOException`.
4. Catch the exception in the calling method.
5. Display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```java
import java.io.IOException;
public class IOExceptionDemo
{
    static void readData() throws IOException
    {
        throw new IOException("Error while reading data.");
    }
    public static void main(String[] args)
    {
        try
        {
            readData();
        }
        catch (IOException e)
        {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Error while reading data.
```

PROGRAM 10 ClassNotFoundException

JAVA.LANG.CLASSNOTFOUNDEXCEPTION

1. QUESTION / OBJECTIVE

> To write a Java program to handle **ClassNotFoundException** when the requested class cannot be found at runtime.

2. STEP-BY-STEP PROCESS

1. Define the class `ClassNotFoundDemo` with the `main()` method.
2. Inside a `try` block, call `Class.forName("UnknownClass")` to dynamically load a non-existent class.
3. When the JVM classloader fails to locate the class, catch the checked `ClassNotFoundException`.
4. Print the error message stating that the class was not found.

3. ALGORITHM

1. Start the program.
2. Try to load a class using `Class.forName()`.
3. Provide a class name that does not exist.
4. Catch the `ClassNotFoundException`.
5. Display the error message.
6. Stop the program.

4. UML CLASS DIAGRAM



5. SOURCE CODE

```
public class ClassNotFoundDemo
{
    public static void main(String[] args)
    {
        try
        {
            Class.forName("UnknownClass");
            System.out.println("Class found.");
        }
        catch (ClassNotFoundException e)
        {
            System.out.println("Exception: Class not found.");
        }
    }
}
```

6. SAMPLE OUTPUT

```
> Exception: Class not found.
```